

MASTER D'EXCELLENCE EN INTELLIGENCE ARTIFICIELLE

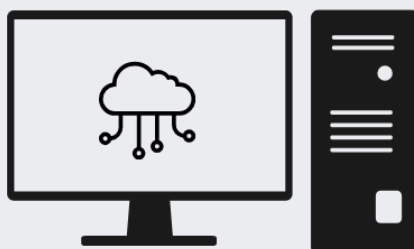
PROJET DE FIN DE MODULE

MISE EN PLACE D'UNE INFRASTRUCTURE DEVOPS AVEC CI/CD, KUBERNETES ET INFRASTRUCTURE AS CODE

2025/2026

Réalisé par :
Assouli Fatima Zahra
Makboul Ouïam

Encadré par :
Pr. F. Benabbou



RÉSUMÉ :

Ce projet s'inscrit dans le cadre de l'adoption croissante des pratiques DevOps, visant à améliorer la collaboration entre les équipes de développement et d'exploitation, tout en automatisant et fiabilisant les processus de déploiement logiciel.

L'objectif principal de ce travail est de concevoir et de mettre en œuvre une infrastructure DevOps complète permettant le déploiement continu d'une application web e-commerce conteneurisée. L'application développée, basée sur le framework Flask, simule une plateforme de vente en ligne offrant des fonctionnalités telles que la gestion des produits, du panier et des utilisateurs.

Afin de garantir la portabilité et la reproductibilité de l'application, celle-ci a été conteneurisée à l'aide de Docker. L'orchestration des conteneurs a été assurée par Kubernetes, en utilisant une distribution légère (k3s), déployée sur une infrastructure cloud hébergée sur AWS.

L'approche Infrastructure as Code a été adoptée à travers l'utilisation de Terraform pour le provisionnement des ressources cloud, ainsi qu'Ansible pour l'automatisation de la configuration des machines et du cluster Kubernetes.

Par ailleurs, une chaîne d'intégration et de déploiement continus (CI/CD) a été mise en place à l'aide de GitLab CI, permettant d'automatiser les étapes de build, de push de l'image Docker et de déploiement sur Kubernetes.

Le résultat final est une infrastructure entièrement automatisée, fiable et reproductible, capable de déployer l'application de manière continue. Ce projet constitue une mise en pratique concrète des principes DevOps dans un environnement réel.

ABSTRACT

This project is part of the growing adoption of DevOps practices, which aim to enhance collaboration between development and operations teams while automating and improving the reliability of software deployment processes.

The main objective of this work is to design and implement a complete DevOps infrastructure that enables the continuous deployment of a containerized e-commerce web application. The developed application, based on the Flask framework, simulates an online shopping platform providing features such as product management, cart handling, and user authentication.

To ensure portability and reproducibility, the application was containerized using Docker. Container orchestration was handled by Kubernetes, using a lightweight distribution (k3s), deployed on a cloud infrastructure hosted on AWS.

The Infrastructure as Code (IaC) approach was adopted through the use of Terraform for provisioning cloud resources, and Ansible for automating system configuration and Kubernetes cluster setup.

In addition, a Continuous Integration and Continuous Deployment (CI/CD) pipeline was implemented using GitLab CI, automating the build process, Docker image publishing, and application deployment on Kubernetes.

The final outcome is a fully automated, reliable, and reproducible infrastructure capable of continuously deploying the application. This project represents a practical implementation of DevOps principles in a real-world environment.

TABLES DE MATIERES :

RÉSUMÉ :	2
ABSTRACT	3
TABLES DE MATIERES :	4
TABLES DE FIGURES:	6
CHAPITE 1 : Introduction	8
1.1 Contexte général :	9
1.2 Problématique :	9
1.3 Objectifs du projet et organisation du rapport :	10
CHAPITRE 2 : Architecture et environnement technique	12
2.1 Vue d'ensemble de l'architecture :	13
2.2 Schéma d'architecture et flux DevOps :	13
2.3 Description détaillée des composants :	14
2.4 Infrastructure Cloud :	15
2.5 Outils et technologies utilisés :	16
Conclusion du Chapitre 2 :	17
Chapitre 3 : Développement et conteneurisation de l'application	18
3.1 Présentation de l'application Swap :	19
3.2 Fonctionnalités principales de Swap :	20
3.3 Tests locaux :	22
3.4 Conteneurisation avec Docker :	24
Conclusion du chapitre :	27
Chapitre 4 : Automatisation et Infrastructure as Code	28
4.1 Introduction à l'Infrastructure as Code (IaC)	29
4.2 Terraform : Provisioning des ressources :	29
4.3 Ansible : Automatisation de la configuration :	31
Conclusion du Chapitre 4 :	33
Chapitre 5 : Orchestration et CI/CD	34
5.1 Introduction à Kubernetes :	35
5.2 Architecture du cluster :	35

5.3 Déploiement de l'application :	35
5.4 Service et exposition de l'application :	36
5.5 Validation du déploiement :	37
5.6 Mise en place du pipeline CI/CD	38
Conclusion du Chapitre 5 :	41
Chapitre 6 : Résultats, analyse et perspectives	42
6.1 Validation de l'infrastructure et de l'application	43
6.2 Tests et captures de validation	43
6.3 Problèmes rencontrés et solutions	44
6.4 Discussion et analyse	45
6.5 Bilan générale	45
6.6 Perspectives d'amélioration	46
Conclusion Générale :	47
Annexes :	48

TABLES DE FIGURES:

Figure 1 : Schéma d'architecture	13
Figure 2 : la page d'accueil de l'application Swap.....	20
Figure 3: : la page gestion de panier de l'application Swap	21
Figure 4: : la page d'authentification de l'application Swap	22
Figure 5: Exécution locale de l'application Flask	23
Figure 6: est des endpoints API via navigateur	24
Figure 7: Dockerfile de l'application	25
Figure 8: Build de l'image Docker	26
Figure 9: Exécution du conteneur Docker	26
Figure 10: login docker	26
Figure 11: Image publiée sur Docker Hub	27
Figure 12 Flux d'automatisation de l'infrastructure DevOps (Terraform & Ansible).....	29
Figure 13: Fichier Terraform (.tf)	30
Figure 14: Initialisation Terraform	30
Figure 15: Création des ressources cloud.....	31
Figure 16: Inventory Ansible.....	31
Figure 17: Playbook Ansible	32
Figure 18: Exécution du playbook	33
Figure 19: Fichier deployment.yaml	36
Figure 20: Pods Kubernetes en cours d'exécution.....	36
Figure 21: Service NodePort.....	37
Figure 22: Application accessible via navigateur (NodePort).....	37
Figure 23 : Extrait du fichier .gitlab-ci.yml utilisé (build).....	39
Figure 24: Extrait du fichier .gitlab-ci.yml utilisé (test)	40
Figure 25: Extrait du fichier .gitlab-ci.yml utilisé (deploy).....	40
Figure 26: Pipeline CI/CD terminé avec succès	41
Figure 27: infrastructure cloud.....	43
Figure 28: Vérification des nœuds Kubernetes	43
Figure 29: Pipeline GitLab CI/CD réussi	44
Figure 30: Image disponible sur Docker Hub	44
Figure 31: structure complete du projet	48
Figure 32: Dockerfile.....	49
Figure 33: Pipeline CI/CD.....	50
Figure 34: main.tf	51
Figure 35: inventory.yml	51
Figure 36: install.yml.....	52
Figure 37: deployment.yaml.....	52
Figure 38 : service.yaml	53
Figure 39: variables CI/CD	53

Liste des tableaux:

Tableau 1 Description des composants d'architecture	14
Tableau 2 Infrastructure Cloud	15
Tableau 3 Outils et technologies utilisés :	16
Tableau 4 : Problèmes rencontrés et solutions	44

CHAPITE 1 : Introduction

1.1 Contexte général :

Dans un contexte marqué par une transformation numérique accélérée, les organisations sont confrontées à une exigence croissante d'agilité, de rapidité et de fiabilité dans le développement et le déploiement des applications. L'émergence du cloud computing et des architectures distribuées a profondément modifié les pratiques traditionnelles de développement logiciel, rendant les cycles de livraison plus courts et plus fréquents.

Cependant, cette évolution s'accompagne de nouveaux défis, notamment en termes de gestion de la complexité, de coordination entre les équipes et de maîtrise des environnements de déploiement.

Les méthodes classiques, souvent cloisonnées entre les équipes de développement et d'exploitation, montrent leurs limites face à ces nouvelles exigences.

Dans ce contexte, l'approche **DevOps** s'est imposée comme une réponse adaptée à ces enjeux. Elle vise à instaurer une collaboration étroite entre les équipes, tout en s'appuyant sur l'automatisation des processus de développement, de test et de déploiement. Grâce à des pratiques telles que l'intégration continue (CI) et le déploiement continu (CD), DevOps permet d'améliorer la qualité des applications, de réduire les délais de mise en production et de minimiser les risques d'erreurs humaines.

Par ailleurs, l'adoption des technologies de conteneurisation, notamment à travers Docker, ainsi que des outils d'orchestration tels que Kubernetes, a permis de standardiser et de simplifier le déploiement des applications dans des environnements variés. Ces technologies jouent un rôle clé dans la mise en œuvre d'infrastructures modernes, scalables et résilientes.

Ainsi, la mise en place d'une infrastructure DevOps complète, intégrant des outils d'automatisation, de conteneurisation et d'orchestration, constitue aujourd'hui un enjeu majeur pour les entreprises souhaitant optimiser leur processus de développement et de déploiement.

1.2 Problématique :

Avec l'évolution des architectures logicielles et l'adoption croissante du cloud computing, le déploiement des applications est devenu une tâche de plus en plus complexe. En effet, il ne s'agit plus seulement de développer une application fonctionnelle, mais également de garantir sa portabilité, sa scalabilité et sa fiabilité dans des environnements distribués.

Dans les approches traditionnelles, les processus de déploiement sont souvent manuels, ce qui les rend sujets aux erreurs, difficilement reproductibles et peu adaptés aux besoins actuels de rapidité et de mise à jour continue. De plus, le manque d'intégration entre les différentes phases du cycle de vie logiciel (développement, test, déploiement) constitue un frein à l'efficacité globale.

Face à ces limites, il devient essentiel de mettre en place une infrastructure capable d'automatiser l'ensemble du processus, depuis la construction de l'application jusqu'à son déploiement en production, tout en assurant sa cohérence et sa stabilité.

1.3 Objectifs du projet et organisation du rapport :

L'objectif principal de ce projet est de concevoir et de mettre en œuvre une infrastructure DevOps complète permettant le déploiement automatisé d'une application conteneurisée dans un environnement cloud.

Pour atteindre cet objectif, plusieurs sous-objectifs ont été définis :

- Développer une application web conteneurisée basée sur une architecture légère et modulaire
- Mettre en place une chaîne d'intégration et de déploiement continu (CI/CD) afin d'automatiser les phases de build, de test et de déploiement
- Déployer et orchestrer l'application à l'aide de Kubernetes afin d'assurer sa scalabilité et sa disponibilité
- Automatiser la création et la configuration de l'infrastructure en utilisant des outils d'Infrastructure as Code tels que Terraform et Ansible

Par ailleurs, ce rapport est structuré de manière à présenter **toutes les étapes du projet de manière logique et progressive** :

1. **Architecture globale et environnement technique** : description de l'architecture du système, des composants utilisés et des technologies sélectionnées pour le projet.
2. **Développement de l'application** : création de l'application web e-commerce, description des fonctionnalités et tests locaux pour valider son fonctionnement.
3. **Conteneurisation avec Docker** : rédaction du Dockerfile, création de l'image et gestion des images pour assurer portabilité et reproductibilité.
4. **Infrastructure as Code avec Terraform et Ansible** : automatisation de la création des machines cloud, configuration des nœuds et mise en place du cluster Kubernetes.
5. **Orchestration avec Kubernetes** : déploiement de l'application sur le cluster, exposition des services et gestion des ressources.
6. **Mise en place de la chaîne CI/CD** : automatisation du cycle complet **build** → **push** → **déploiement via GitLab CI**, garantissant un processus fiable et reproductible.
7. **Analyse des résultats et résolution des problèmes** : validation de l'infrastructure, suivi du déploiement et gestion des problèmes rencontrés (Docker, Git, Kubernetes).
8. **Discussion et perspectives** : évaluation critique du projet, points forts, limites, et propositions d'améliorations futures.

Cette structuration permet au lecteur de suivre le **cheminement complet du projet**, depuis la conception initiale jusqu'à la mise en production automatisée, tout en mettant en évidence les décisions techniques et les bonnes pratiques appliquées.

CHAPITRE 2 : Architecture et environnement technique

2.1 Vue d'ensemble de l'architecture :

Le projet repose sur une approche DevOps intégrée combinant plusieurs outils et technologies pour automatiser le cycle de vie complet de l'application Swap.

L'architecture met en œuvre :

- **Gestion du code source et intégration continue** via GitLab CI/CD.
- **Conteneurisation** de l'application avec Docker pour garantir portabilité et isolation.
- **Orchestration des conteneurs** à l'aide de Kubernetes léger (k3s) pour gérer le déploiement, la scalabilité et la disponibilité des pods.
- **Automatisation de l'infrastructure** et configuration des machines via Terraform et Ansible.
- **Hébergement cloud** sur AWS, assurant l'accès et la stabilité de l'application.

Cette approche modulaire permet de séparer les responsabilités : le pipeline CI/CD déclenche automatiquement la création et le déploiement des images Docker sur Kubernetes, tandis que Terraform et Ansible garantissent une infrastructure stable et reproductible.

2.2 Schéma d'architecture et flux DevOps :

Voici le schéma représentant le flux de déploiement et les interactions entre composants :

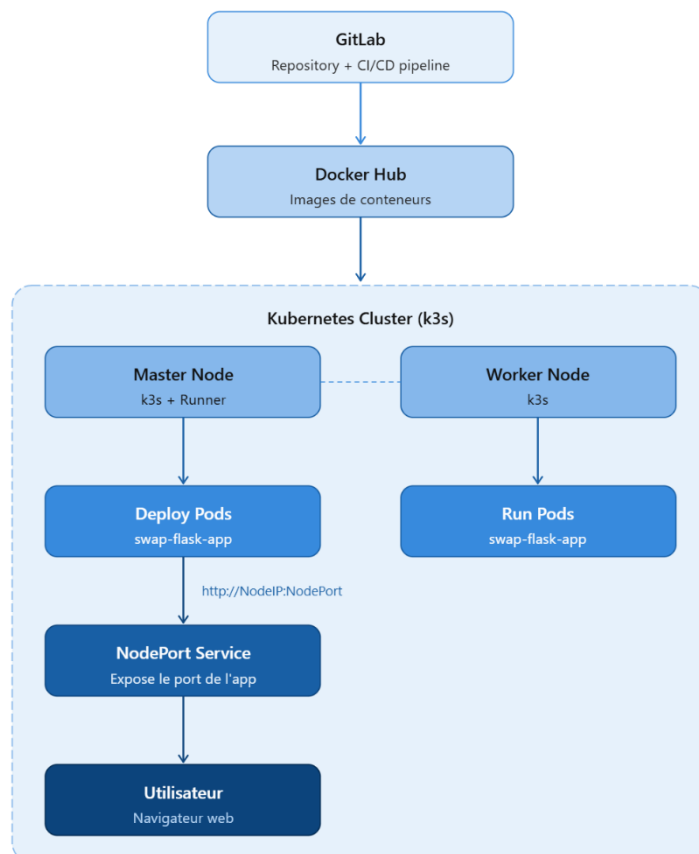


Figure 1 : Schéma d'architecture

Explications du flux :

1. **GitLab CI/CD** : déclenche le pipeline à chaque push, build et teste l'application.
2. **Docker Hub** : stocke les images Docker construites pour être utilisées par le cluster Kubernetes.
3. **Kubernetes (k3s)** :
 - **Master Node** : contrôle le cluster et exécute le GitLab Runner.
 - **Worker Node** : héberge les pods exécutant l'application Swap.
4. **Deployment & Service** : le Deployment crée les pods, et le Service de type NodePort expose l'application sur un port accessible publiquement.
5. **Utilisateur final** : accède à l'application via le navigateur en utilisant l'IP publique du nœud et le NodePort configuré.

2.3 Description détaillée des composants :

Composant	Rôle et responsabilités
Flask	Framework Python pour créer l'API REST, cœur fonctionnel de la plateforme e-commerce Swap.
Docker	Conteneurise l'application, garantissant portabilité, isolation et déploiement rapide.
Kubernetes (k3s)	Orchestration légère des conteneurs ; gère les pods, services et l'équilibrage de charge.
GitLab CI/CD	Automatisation des étapes de build, test, push Docker et déploiement sur Kubernetes.
Terraform	Provisioning des ressources cloud sur AWS (instances EC2, Security Groups, Key Pair).
Ansible	Automatisation de la configuration des machines et du cluster Kubernetes.
AWS	Fournit l'infrastructure cloud : instances virtuelles, réseau et sécurité.

Tableau 1 Description des composants d'architecture

2.4 Infrastructure Cloud :

Pour ce projet, l'infrastructure cloud repose sur **AWS (Amazon Web Services)**, offrant une solution fiable, scalable et sécurisée pour le déploiement des composants DevOps. L'objectif principal est de disposer d'un environnement stable permettant de déployer et tester une application conteneurisée avec un cluster Kubernetes léger (k3s).

Ressource	Description
Instances EC2	Deux machines virtuelles distinctes : • Master : contrôle du cluster et exécution du GitLab Runner • Worker : exécution des pods de l'application
Type de machine	m7i-flex.large, offrant un compromis performant pour la conteneurisation et le déploiement Kubernetes
Système d'exploitation	Ubuntu 22.04 LTS : stabilité, compatibilité avec Docker, k3s et Ansible, et large adoption en production
Réseau	VPC privé avec Security Groups configurés pour autoriser : • SSH pour l'administration • NodePort pour l'accès à l'application • Communication interne entre master et worker
Clés SSH	Accès sécurisé aux instances pour l'installation, la configuration et la maintenance

Tableau 2 Infrastructure Cloud

- La **séparation master/worker** garantit que le contrôle du cluster et l'exécution des pods sont isolés, ce qui reflète une architecture professionnelle.
- Les **Security Groups** assurent la sécurité réseau, en limitant l'accès aux ports nécessaires uniquement.
- L'usage d'**Ubuntu** facilite l'installation et la compatibilité avec les outils DevOps, tout en offrant un environnement stable pour les tests et déploiements.

2.5 Outils et technologies utilisés :

La mise en place de cette infrastructure DevOps repose sur une combinaison d'outils modernes et complémentaires, chacun ayant un rôle précis dans le cycle de vie de l'application.

Catégorie	Outil	Rôle et justification
CI/CD	GitLab CI/CD	Automatisation du pipeline : build, test, push et déploiement continu. Permet un cycle de développement fiable et reproductible.
Conteneurisation	Docker	Isolation de l'application et de ses dépendances, portabilité des environnements et facilité de déploiement sur n'importe quelle machine.
Orchestration	Kubernetes (k3s)	Gestion des conteneurs dans un cluster léger : orchestration des pods, scalabilité et haute disponibilité. Le choix de k3s simplifie l'installation sur AWS tout en offrant les fonctionnalités essentielles.
Infrastructure as Code (IaC)	Terraform	Création et gestion automatique des ressources cloud (instances EC2, VPC, Security Groups). Permet la reproductibilité et la versioning de l'infrastructure.
Configuration et automatisation	Ansible	Déploiement automatique des logiciels et configuration des machines : Docker, k3s et autres dépendances. Simplifie la maintenance et garantit la cohérence des environnements.
Cloud provider	AWS	Fourniture d'infrastructure scalable, fiable et sécurisée. Permet de simuler un environnement professionnel et d'utiliser des services standards de cloud.

Tableau 3 Outils et technologies utilisés :

- Chaque outil est choisi pour sa **stabilité**, sa **compatibilité avec les autres composants** et son adoption dans l'industrie.
- L'intégration de ces outils forme une **chaîne DevOps complète**, allant de l'écriture du code jusqu'au déploiement sur le cloud.
- L'utilisation de **Terraform et Ansible** assure que l'infrastructure et la configuration sont **déclaratives, automatisées et reproductibles**, respectant les meilleures pratiques DevOps.

Conclusion du Chapitre 2 :

Le Chapitre 2 a présenté l'**architecture DevOps** et l'**environnement technique** du projet Swap.

L'intégration de GitLab CI/CD, Docker, Kubernetes (k3s), Terraform, Ansible et AWS assure une **automatisation fiable, une portabilité des applications et une infrastructure stable**. Les rôles de chaque composant sont clairement définis, et l'organisation master/worker garantit la scalabilité et la disponibilité.

Cette architecture fournit une **base solide** pour le développement, la Dockerisation et le déploiement de l'application, qui seront détaillés dans le Chapitre 3.

Chapitre 3 : Développement et conteneurisation de l'application

3.1 Présentation de l'application Swap :

L'application développée est une API REST basée sur le framework Flask, conçue pour simuler une plateforme e-commerce fonctionnelle, nommée **Swap**. Elle a été pensée comme un projet pilote permettant de démontrer l'ensemble du cycle DevOps, depuis le développement jusqu'au déploiement automatisé sur Kubernetes.

Fonctionnalités et rôle de l'application :

Swap permet :

- La consultation d'un catalogue de produits avec affichage des images, noms, prix, réductions et variantes (taille, quantité).
- L'ajout et la gestion des produits dans un panier, avec calcul automatique du sous-total, des taxes et du total général.
- L'interaction via des endpoints REST, facilitant l'intégration dans des systèmes externes et la démonstration des pipelines CI/CD et du déploiement Kubernetes.
- La gestion des utilisateurs pour certaines fonctionnalités sensibles (connexion obligatoire pour le panier et la validation de commandes).

Objectifs principaux :

1. **Représentation fonctionnelle d'un service e-commerce** : L'application illustre toutes les fonctionnalités clés d'une boutique en ligne, tout en restant simple pour un projet académique.
2. **Démonstration du pipeline CI/CD et du déploiement Kubernetes** : Le projet permet de montrer l'automatisation complète, du push de code jusqu'au déploiement de l'application sur le cluster.
3. **Portabilité et isolation** : Grâce à Docker, l'application est entièrement isolée de l'environnement hôte, ce qui garantit une exécution identique sur différents serveurs ou environnements cloud.

Architecture logicielle simplifiée :

- L'API REST expose des **endpoints pour les produits, le panier et les utilisateurs**.
- Les **routes principales** sont implémentées via Flask, avec des fonctions pour récupérer, ajouter ou supprimer des articles.
- Les données peuvent être simulées avec un fichier JSON ou une petite base de données SQLite pour les tests locaux.

Accès et tests locaux :

Avant la conteneurisation, l'application a été testée localement pour s'assurer du bon fonctionnement de toutes les fonctionnalités.

- **Exécution locale** : python run.py
- **URL d'accès** : <http://localhost:5000>

Ces tests locaux permettent de vérifier :

- La cohérence des données du panier et des calculs.
- La navigation fluide et l'affichage des produits.
- La réactivité des endpoints REST, essentielle pour l'intégration dans le pipeline CI/CD et le cluster Kubernetes.

3.2 Fonctionnalités principales de Swap :

L'application Swap reproduit les fonctionnalités essentielles d'une boutique e-commerce et permet de démontrer l'intégration entre le **code**, le **pipeline CI/CD** et le **déploiement Kubernetes**.

1. Page d'accueil et navigation :

- **Barre de recherche** : permet de rechercher tous les produits disponibles (« Search all products... »).
- **Menu principal** : accès aux catégories de produits (Tee, autres vêtements...).
- **Panier** : affiche le nombre d'articles ajoutés et un lien vers le récapitulatif et le paiement.

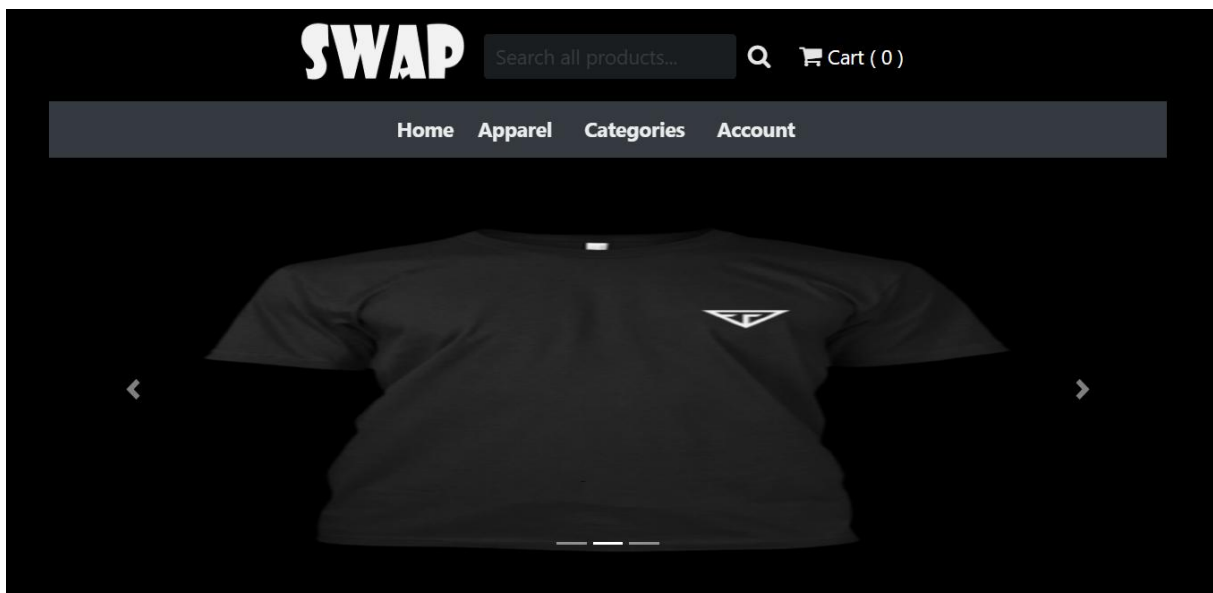


Figure 2 : la page d'accueil de l'application Swap

2. Catalogue de produits :

- Chaque produit affiche :

- Image du produit
- Nom du produit
- Prix actuel et ancien prix si réduction
- Boutons pour sélectionner la **quantité** et la **taille**
- **Section “You May Also Like”** : propose d’autres articles similaires pour inciter à l’achat.

3. Gestion du panier :

- **Affichage détaillé** : liste des articles ajoutés avec image, nom, taille, prix, quantité, réduction et sous-total.
- **Calcul automatique** : total de la commande, taxes incluses.
- **Modification** : possibilité de mettre à jour la quantité ou de supprimer un article.

SWAP Search all products... Q Cart (3)										
Home Apparel Categories Account										
Sr	Image	Name	Size	Price	Quantity	Discount	Subtotal	Update	Remove	
1		Tee	Sizes: Small ▼	\$60.0	1	10% is 6.00	54.00	<button>Update</button>	<button>Remove</button>	
2		Hat	Sizes: Small ▼	\$25.0	1	10% is 2.50	22.50	<button>Update</button>	<button>Remove</button>	
3		Hoodie	Sizes: Small ▼	\$80.0	1	30% is 24.00	56.00	<button>Update</button>	<button>Remove</button>	
				<button>Order now</button>	Tax : \$7.95		Grand Total: \$140.45		<button>Clear Cart</button>	

Figure 3: : la page gestion de panier de l'application Swap

4. Authentification :

- **Login / Signup** : formulaire de connexion avec email et mot de passe.
- **Options** : création de compte, récupération de mot de passe.
- **Protection** : certaines actions nécessitent la connexion.

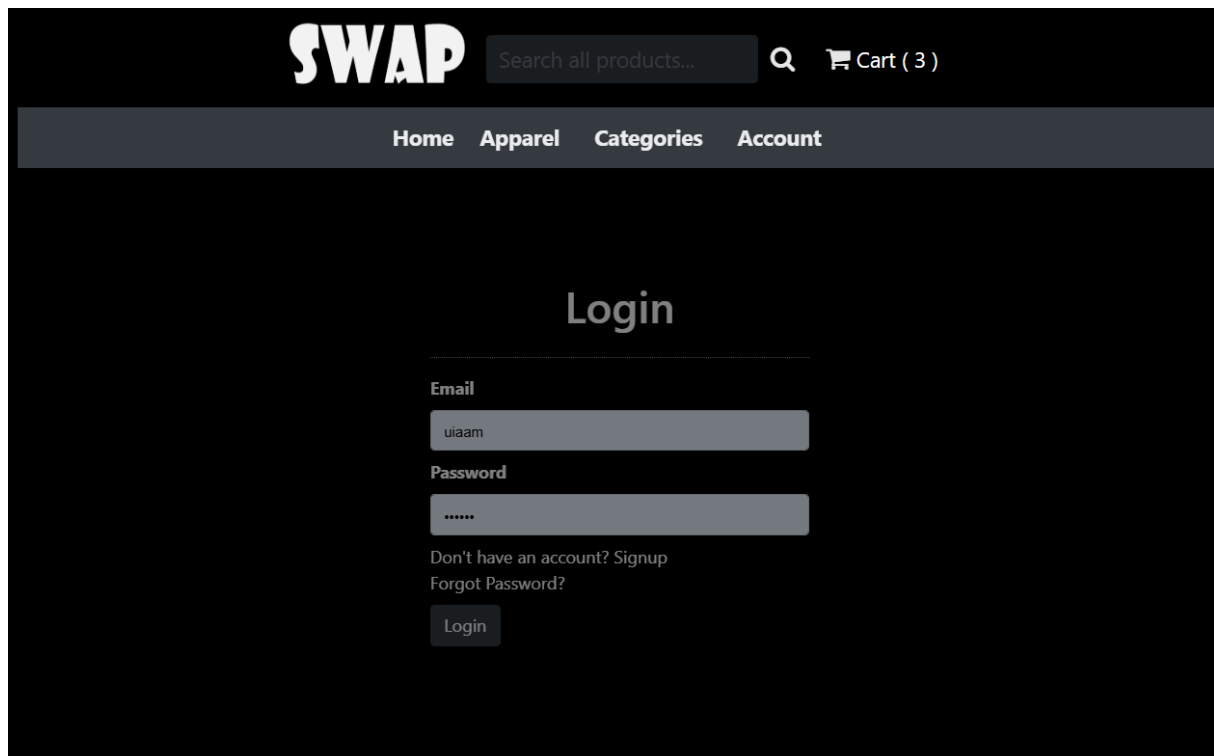


Figure 4: : la page d'authentification de l'application Swap

5. Informations légales et service client :

- Liens en bas de page : Questions / Concerns, Delivery & Returns, Privacy Policy, Terms & Conditions, Contact Us.
- Présentation de l'entreprise : About Swap Inc., Team, Careers.

6. Points techniques visibles :

- Affichage dynamique des prix et des réductions.
- Gestion multi-taille et multi-quantité des produits.
- Calcul automatique du sous-total, des taxes et du total.
- Responsive design : navigation et panier adaptés aux écrans desktop et mobile.
- Intégration possible avec un système de compte utilisateur pour le suivi des commandes.

3.3 Tests locaux :

Avant de déployer l'application sur **Kubernetes**, il est important de vérifier son fonctionnement en local, afin de s'assurer que toutes les fonctionnalités principales (gestion des produits, panier, API REST) sont opérationnelles.

Étapes réalisées :

1. **Installation des dépendances Python** : `pip install -r requirements.txt`
2. **Exécution de l'application Flask en local** : `python run.py`
3. **Accès à l'application via le navigateur** : URL : <http://localhost:5000>

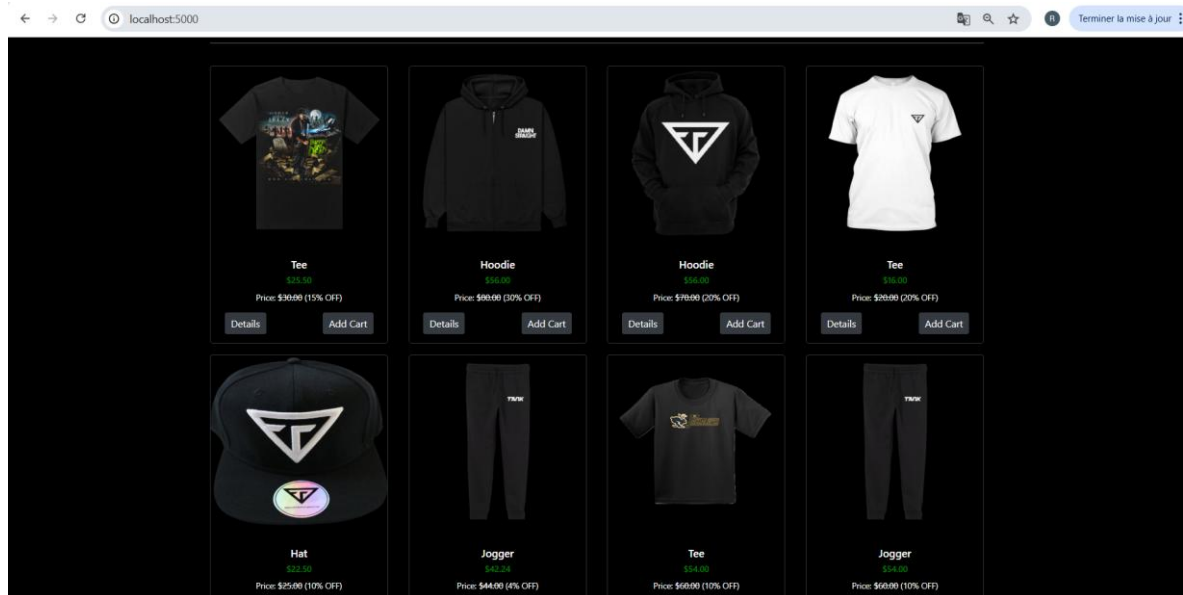


Figure 5: Exécution locale de l'application Flask

4. **Vérifications effectuées :**
 - Page d'accueil et navigation fonctionnelles.
 - Catalogue et panier opérationnels.
 - API REST répond correctement aux requêtes (GET/POST sur les endpoints produits et panier).

```

172.17.0.1 - - [22/Mar/2026 13:59:28] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/4138ba2563dbfd8a9388.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/css/bootstrap.min.css HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/css/custom.css HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/5ec5216f5ae344d1d95f.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/logos/swapOdd.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/90600c39c836245b6e12.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/js/jquery-3.5.1.slim.min.js HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/efe97643265ff2fac691.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/js/bootstrap.min.js HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/js/popper.min.js HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/9c8c6448e4f2037e10ba.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/cfef710cf1eaa2a4ba85.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/f6815c9e23651a6a6044.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/1219dadb03d1236323a2.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/0fc14459cfec6dd4e6e7.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:28] "GET /static/images/1d6b29bcc1a543edcdde.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:29] "GET /static/images/00ebaf38abb1d25844d7.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:31] "GET /product/id/16 HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:32] "GET /static/images/b409d77375cef5a1b82e.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:32] "GET /static/images/6bb54c9fa4bf81407e0e.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:32] "GET /static/images/bfbcfc30b638303d0271.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:32] "GET /static/images/47da6a4e2ace4c87d62a.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:32] "GET /static/images/be76a407c75a45e26e2e.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:32] "GET /static/images/f43c80873547837e2718.png HTTP/1.1" 200 -
172.17.0.1 - - [22/Mar/2026 13:59:34] "POST /addcart HTTP/1.1" 302 -
172.17.0.1 - - [22/Mar/2026 13:59:34] "GET /product/id/16 HTTP/1.1" 200 -

```

Figure 6: est des endpoints API via navigateur

3.4 Conteneurisation avec Docker :

La conteneurisation permet d'isoler l'application et de garantir sa portabilité sur différents environnements, ce qui est fondamental pour le pipeline CI/CD et le déploiement Kubernetes.

Objectifs de la Dockerisation :

- **Portabilité** : l'application peut être exécutée sur n'importe quelle machine disposant de Docker.
- **Isolation** : environnement indépendant des dépendances locales.
- **Déploiement simplifié** : les images Docker peuvent être intégrées directement au pipeline CI/CD et déployées sur Kubernetes.

Dockerfile :

- **Image de base** : python:3.9-slim
- **Installation des dépendances** depuis requirements.txt
- **Copie du code source** dans le conteneur
- **Exécution de l'application** via python run.py

```
Dockerfile X
swap-app > Dockerfile >...
1  # Dockerfile pour swap-flask-app
2  FROM python:3.9-slim
3
4  # Installer outils nécessaires
5  RUN apt-get update && apt-get install -y \
6      build-essential \
7      python3-dev \
8      libffi-dev \
9      && rm -rf /var/lib/apt/lists/*
10
11 WORKDIR /app
12
13 # Copier les fichiers
14 COPY requirements.txt /app/
15 COPY . /app/
16
17 # Installer les dépendances
18 RUN pip install --upgrade pip
19 RUN pip install --no-cache-dir -r requirements.txt
20
21 EXPOSE 5000
22
23 CMD ["python", "run.py"]
```

Figure 7: Dockerfile de l'application

Gestion des images Docker

1. Build de l'image :

`docker build -t uiaam/swap-flask-app:latest .`

```
(env) PS C:\Users\hp\devops-project\docker> docker build -t uiaam/swap-flask-app:latest C:\Users\hp\devops-project\docker\swap-app
[+] Building 24.8s (13/13) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 510B                      0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.2s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                            0.0s
=> [1/7] FROM docker.io/library/python:3.9-slim@sha256:2d97f69 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f69 0.0s
=> [internal] load build context                        2.0s
=> => transferring context: 32.99MB                      2.0s
=> CACHED [2/7] RUN apt-get update && apt-get install -y    b 0.0s
=> CACHED [3/7] WORKDIR /app                             0.0s
=> CACHED [4/7] COPY requirements.txt /app/               0.0s
=> [5/7] COPY . /app/                                    0.3s
=> [6/7] RUN pip install --upgrade pip                   4.6s
=> [7/7] RUN pip install --no-cache-dir -r requirements.txt 14.1s
=> exporting to image                                    2.4s
=> => exporting layers                                    1.5s
=> => exporting manifest sha256:db63c8f174ee436241766d14363a0e 0.0s
=> => exporting config sha256:5d164e995eccc9f941857242c3187701 0.0s
=> => exporting attestation manifest sha256:6c46eacfc52b786c9c 0.0s
=> => exporting manifest list sha256:a32a7bd1fa0da2b4249332034 0.0s
=> => naming to docker.io/uiaam/swap-flask-app:latest      0.0s
=> => unpacking to docker.io/uiaam/swap-flask-app:latest  0.8s
```

Figure 8: Build de l'image Docker

2. Exécution en local pour test :

docker run -p 5000:5000 uiaam/swap-flask-app:latest

```
(env) PS C:\Users\hp\devops-project\docker> docker run -d -p 5000:5000 uiaam/swap-flask-app:latest
13bc7f25be0b86feb96639225053323ea0764524509f1ba564886cc3637c6420
(env) PS C:\Users\hp\devops-project\docker> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
13bc7f25be0b   uiaam/swap-flask-app:latest         "python run.py"         5 seconds ago Up 4 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   musing_antonelli
(env) PS C:\Users\hp\devops-project\docker>
```

Figure 9: Exécution du conteneur Docker

3. Push sur Docker Hub pour CI/CD :

docker login

```
(env) PS C:\Users\hp\devops-project\docker> echo dckr_pat_rsZ71--D98XL70IcjfyTDq36Mpw | docker login
docker.io -u uiaam --password-stdin
Login Succeeded
```

Figure 10: login docker

docker push uiaam/swap-flask-app:latest

```
(env) PS C:\Users\hp\devops-project\docker> docker push uiaam/swap-flask-app:latest
The push refers to repository [docker.io/uiaam/swap-flask-app]
0e36e5b0761a: Pushed
ea56f685404a: Layer already exists
4bb3902a7bda: Layer already exists
60d39c6d5485: Layer already exists
dfc4a9a4b5a1: Layer already exists
d988525710ce: Pushed
b3ec39b36ae8: Layer already exists
fc7443084902: Layer already exists
57907fa0e589: Pushed
4b6320e3394d: Pushed
38513bd72563: Layer already exists
latest: digest: sha256:a32a7bd1fa0da2b424933203489a948871c5db8f65f25a93ea43f378c87a369f size: 856
(env) PS C:\Users\hp\devops-project\docker>
```

Figure 11: Image publiée sur Docker Hub

Conclusion du chapitre :

Ce chapitre a présenté le développement de l'application Swap, ses fonctionnalités principales et les tests locaux validant son fonctionnement.

La Dockerisation a permis d'isoler l'application, d'assurer sa portabilité et de créer une image stable prête pour le déploiement sur Kubernetes via le pipeline CI/CD.

En résumé : l'application est fonctionnelle, contenerisée et prête à être intégrée dans l'infrastructure DevOps automatisée décrite dans les chapitres suivants.

Chapitre 4 : Automatisation et Infrastructure as Code

4.1 Introduction à l'Infrastructure as Code (IaC)

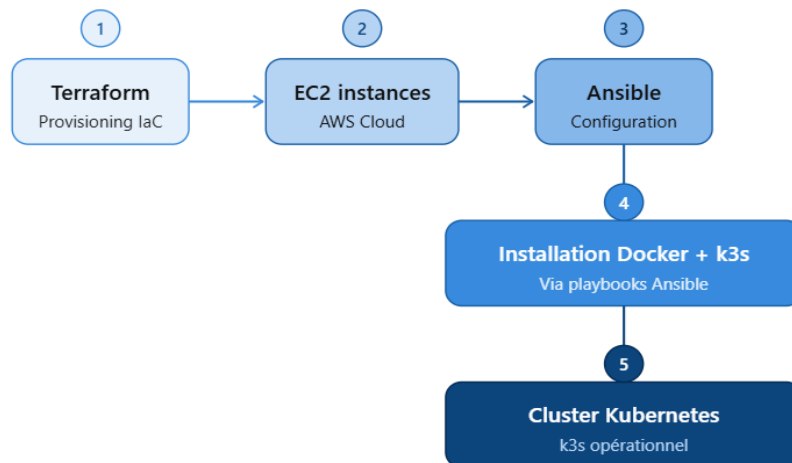


Figure 12 Flux d'automatisation de l'infrastructure DevOps (Terraform & Ansible)

L'Infrastructure as Code (IaC) est une approche qui consiste à décrire l'infrastructure sous forme de code, plutôt que de la configurer manuellement. Cela permet :

- **Reproductibilité** : chaque déploiement est identique.
- **Automatisation** : plus besoin d'interventions manuelles pour créer ou modifier des ressources.
- **Versioning** : l'infrastructure peut être versionnée et auditable comme du code.

Dans ce projet, nous avons utilisé deux outils complémentaires :

- **Terraform** : pour le provisionnement de l'infrastructure cloud.
- **Ansible** : pour la configuration et l'automatisation des machines et du cluster Kubernetes.

4.2 Terraform : Provisioning des ressources :

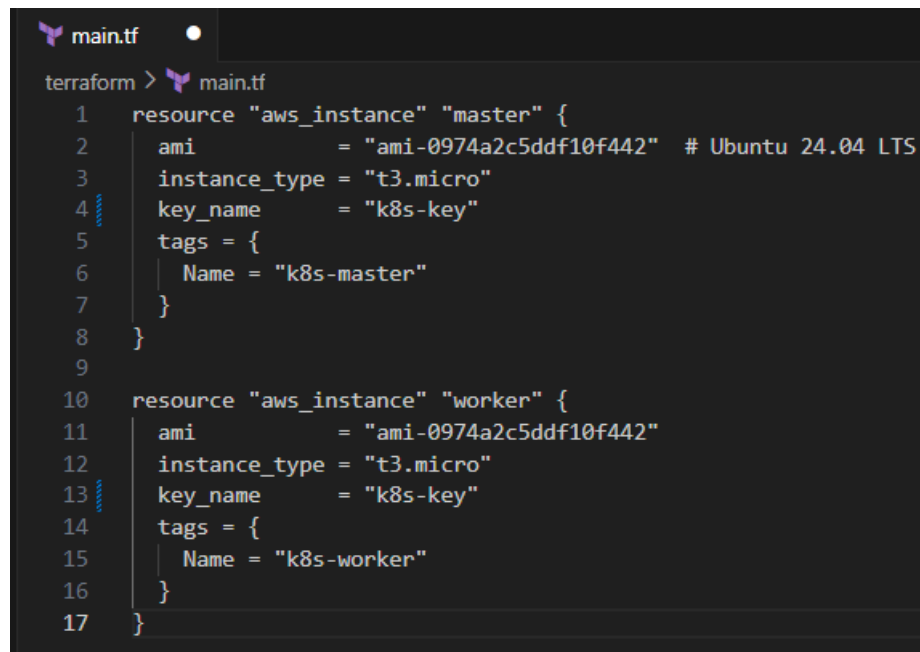
Terraform est un outil open source qui permet de définir, créer et gérer l'infrastructure cloud de manière déclarative.

Rôle dans le projet :

- Créer et gérer les instances AWS EC2 (Master et Worker).
- Configurer les Security Groups pour sécuriser l'accès SSH et les ports d'application.
- Générer les Key Pairs SSH pour un accès sécurisé.
- Créer un environnement cloud reproductible et versionné.

Étapes réalisées :

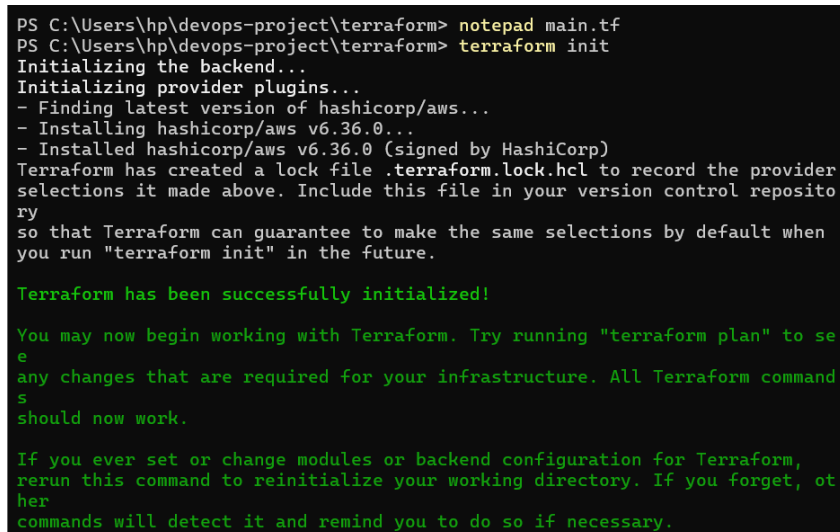
1. Écriture des fichiers .tf : définition des ressources (instances, réseaux, sécurité).



```
main.tf
terraform > main.tf
1 resource "aws_instance" "master" {
2     ami           = "ami-0974a2c5ddf10f442" # Ubuntu 24.04 LTS
3     instance_type = "t3.micro"
4     key_name      = "k8s-key"
5     tags = {
6         Name = "k8s-master"
7     }
8 }
9
10 resource "aws_instance" "worker" {
11     ami           = "ami-0974a2c5ddf10f442"
12     instance_type = "t3.micro"
13     key_name      = "k8s-key"
14     tags = {
15         Name = "k8s-worker"
16     }
17 }
```

Figure 13: Fichier Terraform (.tf)

2. Initialisation du projet Terraform : **terraform init**



```
PS C:\Users\hp\devops-project\terraform> notepad main.tf
PS C:\Users\hp\devops-project\terraform> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.36.0...
- Installed hashicorp/aws v6.36.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Figure 14: Initialisation Terraform

3. Prévisualisation des ressources à créer : **terraform plan**
4. Création effective des ressources : **terraform apply**

```

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.worker: Creating...
aws_instance.master: Creating...
aws_instance.worker: Still creating... [00m10s elapsed]
aws_instance.master: Still creating... [00m10s elapsed]
aws_instance.master: Creation complete after 14s [id=i-03d3561b37fbf48c4]
aws_instance.worker: Creation complete after 14s [id=i-0455c9ae5c6b6325c]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

```

Figure 15: Création des ressources cloud

4.3 Ansible : Automatisation de la configuration :

Ansible est un outil d'automatisation qui permet de déployer et configurer des machines à distance via des playbooks YAML, sans installation d'agent sur les machines.

Rôle dans le projet :

- Installer et configurer Docker sur toutes les machines.
- Installer et configurer k3s pour former le cluster Kubernetes Master/Worker.
- Appliquer automatiquement les paramètres réseau et configurations système.
- Assurer la cohérence entre les environnements.

Organisation et étapes :

1. **Inventory** : fichier listant les machines (Master et Worker) et leurs rôles.

```

≡ inventory ×
ansible > ≡ inventory
1  [masters]
2  master1 ansible_host=16.16.65.232 ansible_user=ubuntu ansible_ssh_private_key_file=/root/.ssh/k8s-key.pem
3
4  [workers]
5  worker1 ansible_host=56.228.15.238 ansible_user=ubuntu ansible_ssh_private_key_file=/root/.ssh/k8s-key.pem
6

```

Figure 16: Inventory Ansible

2. **Playbooks** : scripts automatisant les installations et configurations.

```
! install.yml ●
ansible > ! install.yml
1  - name: Configure servers
2    hosts: all
3    become: yes
4
5    tasks:
6
7      - name: Update apt
8        apt:
9          update_cache: yes
10
11     - name: Install required packages
12       apt:
13         name:
14           - docker.io
15           - python3-pip
16           - curl
17         state: present
18
19     - name: Start Docker
20       service:
21         name: docker
22         state: started
23         enabled: yes
24
25     - name: Add ubuntu to docker group
26       user:
27         name: ubuntu
28         groups: docker
29         append: yes
30
31     - name: Install k3s
32       shell: curl -sfl https://get.k3s.io | sh -
```

Figure 17: Playbook Ansible

3. Exécution des playbooks : `ansible-playbook -i inventory setup_cluster.yml`

```

root@DESKTOP-OUIAM0704004:/mnt/c/Users/hp/devops-project/ansible# ansible all -i in
ventory -m ping
worker1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
master1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
root@DESKTOP-OUIAM0704004:/mnt/c/Users/hp/devops-project/ansible# ansible-playbook
-i inventory install.yml

PLAY [Install Docker] *****

TASK [Gathering Facts] *****
ok: [worker1]
ok: [master1]

TASK [Update apt] *****
changed: [worker1]

```

Figure 18: Exécution du playbook

Conclusion du Chapitre 4 :

Ce chapitre a mis en évidence l'importance de l'Infrastructure as Code et de l'automatisation dans la mise en place d'un environnement DevOps.

L'utilisation de **Terraform** a permis de provisionner une infrastructure cloud reproductible et sécurisée, tandis que **Ansible** a assuré la configuration automatique des machines et du cluster Kubernetes.

Ainsi, l'infrastructure obtenue est cohérente, automatisée et prête à accueillir le déploiement continu de l'application dans les chapitres suivants.

Chapitre 5 : Orchestration et CI/CD

5.1 Introduction à Kubernetes :

Kubernetes est une plateforme open source d'orchestration de conteneurs qui permet d'automatiser le déploiement, la gestion et la mise à l'échelle des applications conteneurisées.

Dans ce projet, nous avons utilisé **k3s**, une version légère de Kubernetes, adaptée aux environnements cloud simples et aux projets académiques. Ce choix permet de bénéficier des fonctionnalités essentielles de Kubernetes tout en simplifiant son installation et sa configuration.

L'utilisation de Kubernetes permet notamment de :

- Déployer automatiquement l'application sous forme de pods
- Assurer la disponibilité et la résilience
- Gérer facilement la scalabilité
- Simplifier la mise à jour de l'application

5.2 Architecture du cluster :

Le cluster Kubernetes mis en place repose sur une architecture simple composée de deux nœuds:

Master Node

- Gère le cluster et les ressources
- Planifie les déploiements
- Héberge le **control plane**
- Exécute également le **GitLab Runner**

Worker Node

- Exécute les **pods** de l'application
- Héberge les conteneurs Docker

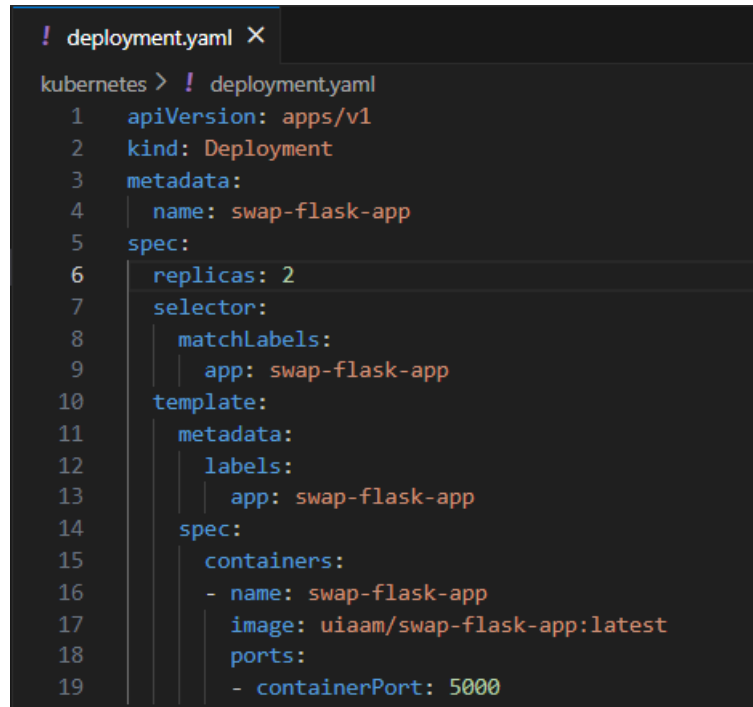
Cette séparation permet une meilleure organisation et reflète une architecture utilisée en environnement professionnel.

5.3 Déploiement de l'application :

Le déploiement de l'application est réalisé via un fichier YAML appelé `deployment.yaml`, qui décrit l'état souhaité de l'application dans le cluster.

Rôle du Deployment :

- Créer et gérer les pods
- Assurer la réplication (plusieurs instances)
- Garantir que l'application reste disponible



```

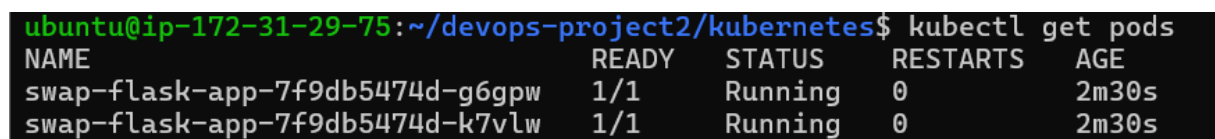
! deployment.yaml X
kubernetes > ! deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: swap-flask-app
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: swap-flask-app
10   template:
11     metadata:
12       labels:
13         app: swap-flask-app
14     spec:
15       containers:
16       - name: swap-flask-app
17         image: uiaam/swap-flask-app:latest
18         ports:
19         - containerPort: 5000

```

Figure 19: Fichier deployment.yaml

Contenu principal :

- Image Docker : uiaam/swap-flask-app:latest
- Nombre de replicas
- Port de l'application



```

ubuntu@ip-172-31-29-75:~/devops-project2/kubernetes$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
swap-flask-app-7f9db5474d-g6gpw     1/1     Running   0           2m30s
swap-flask-app-7f9db5474d-k7v1w     1/1     Running   0           2m30s

```

Figure 20: Pods Kubernetes en cours d'exécution

5.4 Service et exposition de l'application :

Pour rendre l'application accessible depuis l'extérieur, un Service Kubernetes est utilisé.

Type utilisé : NodePort

Le service NodePort permet :

- D'exposer l'application sur un port du nœud
- D'accéder à l'application via l'adresse IP publique

```
ubuntu@ip-172-31-29-75:~/devops-project2/kubernetes$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
hello-world         NodePort    10.43.85.106  <none>         80:30338/TCP     6d18h
kubernetes           ClusterIP   10.43.0.1     <none>         443/TCP          6d19h
swap-flask-app-service NodePort    10.43.62.112  <none>         5000:30007/TCP   4d16h
```

Figure 21: Service NodePort

Fonctionnement :

- Kubernetes assigne un port (30007)
- L'application devient accessible via :
- Navigateur montrant l'application déployée

5.5 Validation du déploiement :

Après le déploiement, plusieurs vérifications ont été réalisées :

- Vérification des pods en cours d'exécution
- Vérification du service exposé
- Accès réussi à l'application via le navigateur

Ces validations confirment que l'application est correctement **orchestrée et accessible** via Kubernetes.

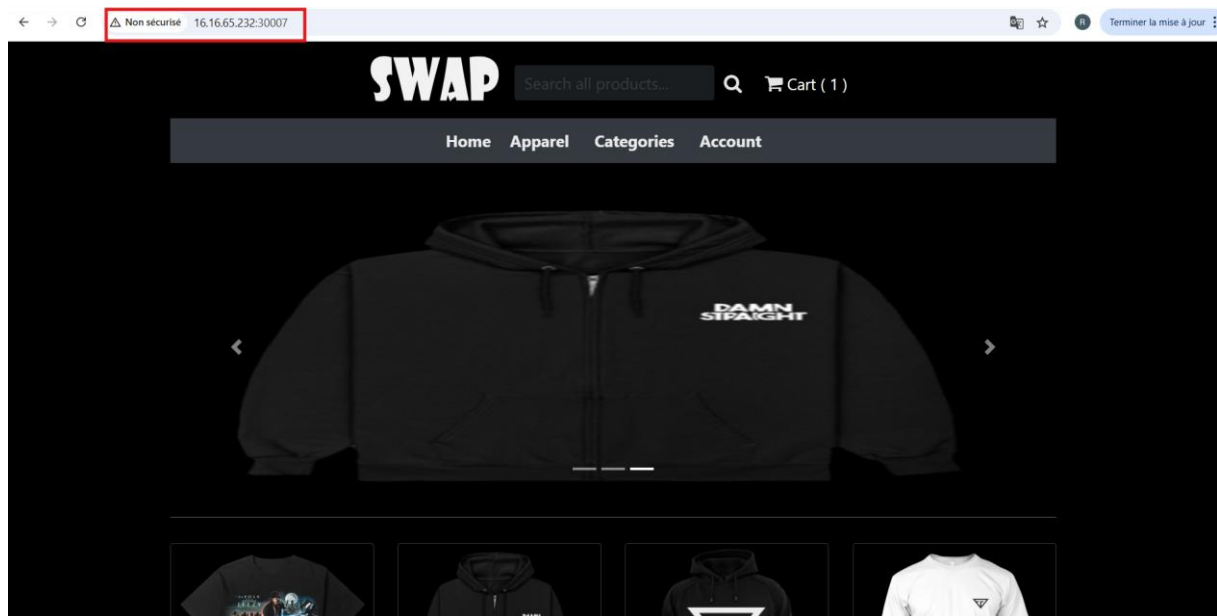


Figure 22: Application accessible via navigateur (NodePort)

5.6 Mise en place du pipeline CI/CD

La mise en place d'un pipeline d'intégration et de déploiement continu (CI/CD) constitue un élément central de l'approche DevOps adoptée dans ce projet. Elle permet d'automatiser l'ensemble du cycle de vie de l'application, depuis le développement jusqu'au déploiement sur le cluster Kubernetes.

Pour ce projet, nous avons utilisé **GitLab CI/CD**, qui déclenche automatiquement les étapes du pipeline à chaque modification du code source (push sur la branche `main`).

1. Fonctionnement du pipeline

Le pipeline CI/CD se compose de trois phases principales :

1. Build :

- Construction de l'image Docker de l'application `swap-flask-app`.
- Authentification auprès de Docker Hub et push de l'image avec le tag `latest`.

2. Test :

- Exécution des tests unitaires avec `pytest`.
- Vérification de la qualité et du bon fonctionnement du code avant déploiement.

3. Deploy :

- Déploiement automatique sur Kubernetes via `kubectl apply`.
- Redémarrage du déploiement pour appliquer la nouvelle version de l'application.

Cette automatisation permet de :

- Garantir un déploiement rapide et reproductible
- Réduire les interventions manuelles et les erreurs humaines
- Maintenir l'application toujours à jour

2. Configuration du pipeline (`.gitlab-ci.yml`)

Le pipeline est défini dans le fichier `.gitlab-ci.yml` situé à la racine du projet.

Structure principale :

- stages : `build` → `test` → `deploy`
- variables : définition de `IMAGE_NAME`, `IMAGE_TAG`, et configuration Docker
- Étapes scriptées pour `build`, `push`, `test` et déploiement Kubernetes

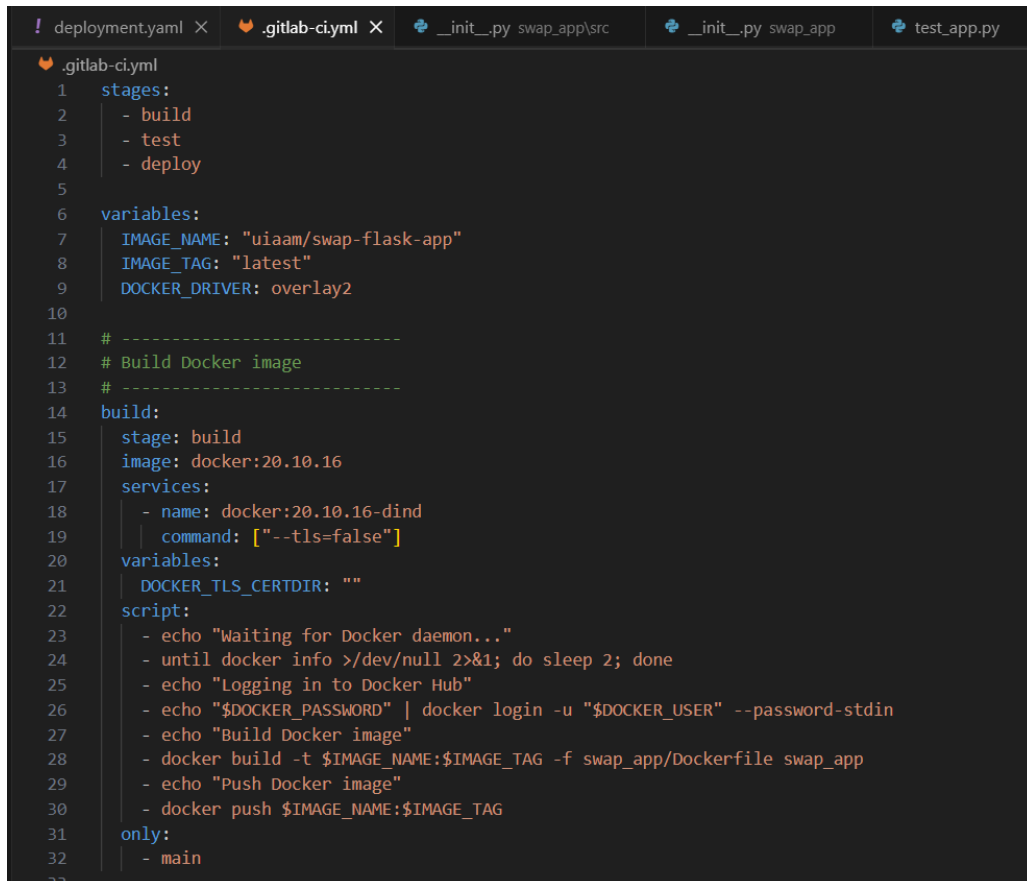
Contenu résumé :

- **Build** : docker build et push

Docker daemon est initialisé dans le runner GitLab (docker:dind)

L'image est construite à partir du Dockerfile dans swap_app/

L'image est ensuite poussée sur Docker Hub (\$IMAGE_NAME:\$IMAGE_TAG)



```
! deployment.yaml X .gitlab-ci.yml X _init_.py swap_app\src _init_.py swap_app test_app.py
.gitlab-ci.yml
1  stages:
2    - build
3    - test
4    - deploy
5
6  variables:
7    IMAGE_NAME: "uiaam/swap-flask-app"
8    IMAGE_TAG: "latest"
9    DOCKER_DRIVER: overlay2
10
11  # -----
12  # Build Docker image
13  # -----
14  build:
15    stage: build
16    image: docker:20.10.16
17    services:
18      - name: docker:20.10.16-dind
19        command: ["--tls=false"]
20    variables:
21      DOCKER_TLS_CERTDIR: ""
22    script:
23      - echo "Waiting for Docker daemon..."
24      - until docker info >/dev/null 2>&1; do sleep 2; done
25      - echo "Logging in to Docker Hub"
26      - echo "$DOCKER_PASSWORD" | docker login -u "$DOCKER_USER" --password-stdin
27      - echo "Build Docker image"
28      - docker build -t $IMAGE_NAME:$IMAGE_TAG -f swap_app/Dockerfile swap_app
29      - echo "Push Docker image"
30      - docker push $IMAGE_NAME:$IMAGE_TAG
31    only:
32      - main
33
```

Figure 23 : Extrait du fichier .gitlab-ci.yml utilisé (build)

- **Test** : exécution de pytest

L'étape test exécute les tests unitaires via pytest :

Installation des dépendances Python (requirements.txt)

Tests exécutés avec un seul échec maximum (--maxfail=1)

Dossier swap_app ajouté au PYTHONPATH pour gérer les imports

Cette étape permet de valider le code avant tout déploiement et d'éviter d'envoyer des erreurs en production.

```

34 # -----
35 # Test avec pytest
36 # -----
37 test:
38   stage: test
39   image: python:3.9
40   before_script:
41     - pip install --upgrade pip
42     - pip install -r swap_app/requirements.txt
43     - pip install pytest
44     # Ajouter le dossier swap_app au PYTHONPATH pour les imports
45     - export PYTHONPATH="$CI_PROJECT_DIR/swap_app:$PYTHONPATH"
46   script:
47     - pytest tests --maxfail=1 --disable-warnings -q
48   only:
49     - main
50

```

Figure 24: Extrait du fichier .gitlab-ci.yml utilisé (test)

- **Deploy** : application sur le cluster Kubernetes avec kubectl

L'étape deploy exécute :

1. Configuration du fichier kubeconfig à partir de la variable CI \$KUBECONFIG_CONTENT
2. Vérification de la connexion au cluster : kubectl get nodes
3. Application des fichiers Kubernetes :

```

51 # -----
52 # Deploy sur Kubernetes
53 # -----
54 deploy:
55   stage: deploy
56   image:
57     name: bitnami/kubectl:latest
58     entrypoint: [""]
59   needs: [build, test]
60   script:
61     - echo "Setup kubeconfig"
62     - mkdir -p ~/.kube
63     - echo "$KUBECONFIG_CONTENT" > ~/.kube/config
64     - export KUBECONFIG=~/.kube/config
65     - echo "Test cluster connection"
66     - kubectl get nodes
67     - echo "Deploy app"
68     - kubectl apply -f kubernetes/deployment.yaml
69     - kubectl apply -f kubernetes/service.yaml
70     - kubectl rollout restart deployment swap-flask-app
71   only:
72     - main

```

Figure 25: Extrait du fichier .gitlab-ci.yml utilisé (deploy)

3. Résultat du pipeline

Le pipeline s'exécute avec succès sur la branche main :

- Toute modification du code déclenche le build, les tests et le déploiement
- L'infrastructure reste cohérente et l'application disponible en continu

Ouiam Makboul / devops-flask-app

Created on

March 17, 2026

main

devops-flask-app

+

Find file

Code

 **init a swap**
Ouiam authored 7 hours ago

✓

cbd70d57

History

Name	Last commit	Last update
📁 ansible	Add ansible + terraform	3 days ago
📁 kubernetes	modifie dep	10 hours ago
📁 swap_app	init a swap	7 hours ago
📁 terraform	Ignore Terraform state files	3 days ago
📁 tests	init a swap	7 hours ago
🔥 .gitignore	Ignore Terraform state files	3 days ago
🔥 .gitlab-ci.yml	modif gitlab swap_app	8 hours ago

Figure 26: Pipeline CI/CD terminé avec succès

Conclusion du Chapitre 5 :

L'intégration du pipeline CI/CD a permis d'automatiser complètement le processus de déploiement de l'application.

Cette approche illustre les principes DevOps en garantissant :

- Livraison continue et rapide
- Réduction des erreurs humaines
- Reproductibilité et fiabilité du système

Chapitre 6 : Résultats, analyse et perspectives

Ce chapitre présente les résultats obtenus à l'issue de la mise en place de l'infrastructure DevOps, ainsi que la validation du système, une analyse critique de la solution proposée et les perspectives d'amélioration.

6.1 Validation de l'infrastructure et de l'application

La validation du système a été réalisée en vérifiant le bon fonctionnement de l'ensemble des composants mis en place, depuis l'infrastructure cloud jusqu'au déploiement de l'application.

Les résultats obtenus confirment que :

- L'infrastructure cloud a été correctement provisionnée, avec des instances opérationnelles.

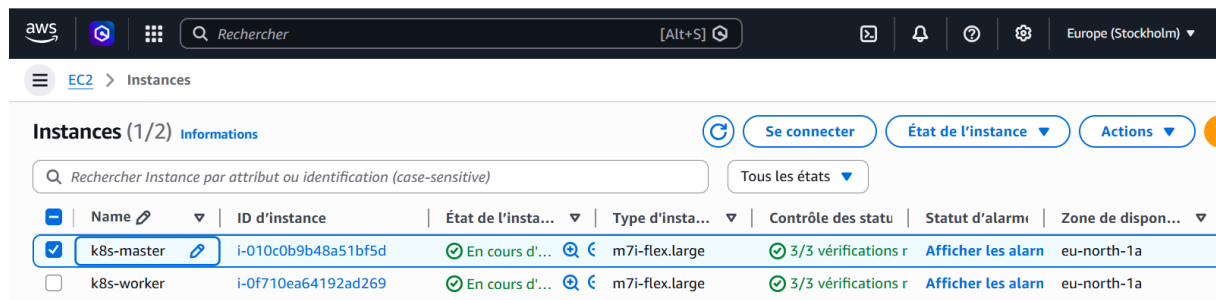


Figure 27: infrastructure cloud

- Le cluster Kubernetes est fonctionnel, avec une communication établie entre le nœud master et le nœud worker.
- L'application a été déployée avec succès et est accessible via un service de type NodePort.
- Le pipeline CI/CD s'exécute automatiquement à chaque modification du code, sans erreurs.

```
ubuntu@ip-172-31-29-75:~/devops-project2/kubernetes$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
ip-172-31-19-217    Ready    <none>   6d18h v1.34.5+k3s1
ip-172-31-29-75     Ready    control-plane 6d19h v1.34.5+k3s1
```

Figure 28: Vérification des nœuds Kubernetes

Ces éléments démontrent la cohérence et la stabilité de l'environnement mis en place.

6.2 Tests et captures de validation

Afin de garantir le bon fonctionnement du système, plusieurs tests ont été effectués à différents niveaux de l'architecture.

- **Test du pipeline CI/CD :** chaque push déclenche automatiquement le pipeline, incluant le build, le push de l'image Docker et le déploiement.
- **Test d'accès à l'application :** l'application est accessible via l'adresse IP publique et le NodePort configuré.

- **Vérification Kubernetes** : les pods sont correctement déployés et restent en état *Running*.
- **Test de mise à jour continue** : toute modification du code entraîne une mise à jour automatique de l'application déployée.

Ces tests confirment le bon fonctionnement de l'intégration continue et du déploiement continu.

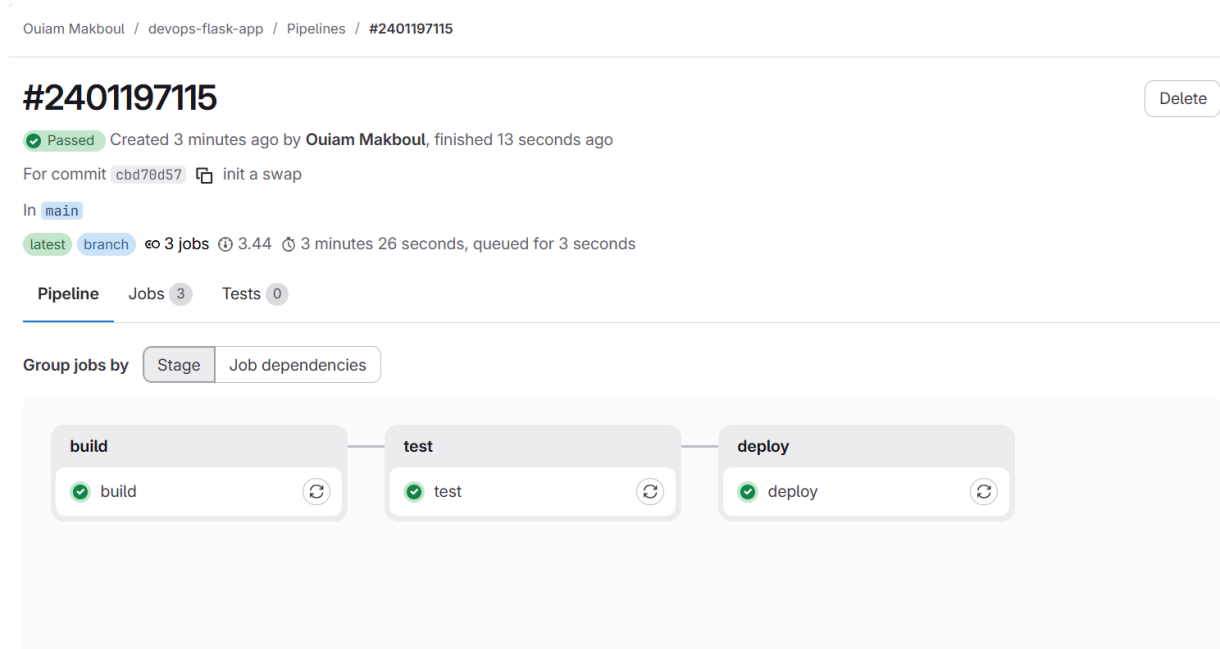


Figure 29: Pipeline GitLab CI/CD réussi

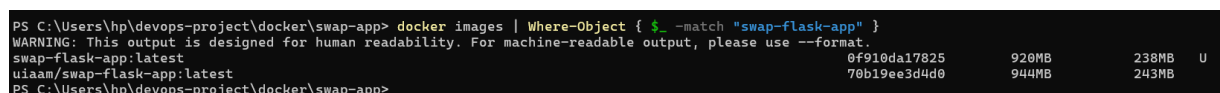


Figure 30: Image disponible sur Docker Hub

6.3 Problèmes rencontrés et solutions

Au cours de la réalisation du projet, plusieurs difficultés techniques ont été rencontrées. Leur résolution a permis de renforcer la robustesse de l'infrastructure et d'acquérir des compétences pratiques en debugging DevOps.

Tableau 4 : Problèmes rencontrés et solutions

Problème	Solution
Erreurs lors du build/push Docker	Vérification du Dockerfile et authentification Docker Hub
Dysfonctionnement du pipeline CI/CD	Correction et optimisation du fichier .gitlab-ci.yml
Accès Kubernetes non fonctionnel	Configuration correcte du fichier kubeconfig

Difficultés d'installation de k3s	Reconfiguration du cluster et vérification réseau
Accès externe bloqué (NodePort)	Ouverture des ports dans les Security Groups

Ces incidents ont été résolus de manière progressive, contribuant à une meilleure compréhension des interactions entre les différents outils.

6.4 Discussion et analyse

Points forts

- Automatisation complète du cycle de vie applicatif (CI/CD + IaC)
- Architecture cohérente et proche des environnements professionnels
- Intégration réussie d'outils modernes du DevOps
- Reproductibilité et fiabilité de l'infrastructure

Limites

- Absence de monitoring
- Sécurité basique
- Scalabilité limitée

Comparaison technique : k3s vs kubeadm

Le choix de k3s s'explique par :

- Sa légèreté et sa simplicité d'installation
- Son adaptation aux environnements à ressources limitées
- Sa rapidité de mise en œuvre

À l'inverse, une solution comme kubeadm offre plus de flexibilité et de fonctionnalités, mais avec une complexité plus élevée.

Ainsi, k3s représente un compromis pertinent pour un projet académique.

6.5 Bilan générale

Ce projet a permis de mettre en œuvre une chaîne DevOps complète, couvrant l'ensemble du cycle de vie d'une application, du développement jusqu'au déploiement automatisé dans un environnement cloud.

Les compétences développées incluent :

- La conteneurisation avec Docker
- L'orchestration avec Kubernetes

- La mise en place de pipelines CI/CD
- L'utilisation de Terraform et Ansible pour l'Infrastructure as Code

Cette expérience constitue une base solide pour aborder des architectures plus complexes en environnement professionnel.

6.6 Perspectives d'amélioration

Plusieurs axes d'amélioration peuvent être envisagés afin d'enrichir et de professionnaliser davantage l'infrastructure :

- Intégration d'un système de monitoring (Prometheus, Grafana)
- Mise en place de mécanismes de sécurité avancés (HTTPS, RBAC, gestion des secrets)
- Implémentation de la scalabilité automatique via HPA (Horizontal Pod Autoscaler)
- Amélioration du pipeline CI/CD (tests automatisés, gestion multi-environnements)

Ces perspectives ouvrent la voie à une évolution vers une architecture plus robuste, sécurisée et adaptée aux exigences industrielles.

Conclusion Générale :

Ce projet a permis de concevoir et de mettre en œuvre une **infrastructure DevOps complète**, allant du développement d'une application web jusqu'à son déploiement automatisé dans un environnement cloud. L'objectif principal, consistant à automatiser le cycle de vie d'une application conteneurisée, a été atteint grâce à l'intégration cohérente de plusieurs outils modernes et complémentaires.

Dans un premier temps, une application e-commerce simulée (**Swap**) a été développée et testée localement afin de valider ses fonctionnalités. Elle a ensuite été conteneurisée avec Docker, garantissant sa portabilité et son indépendance vis-à-vis de l'environnement d'exécution.

Par la suite, l'infrastructure cloud a été entièrement automatisée à l'aide de Terraform, permettant la création reproductible des ressources sur AWS. Ansible a ensuite été utilisé pour configurer les machines et déployer un cluster Kubernetes (k3s), assurant ainsi une base stable et cohérente pour l'orchestration des conteneurs.

L'intégration de Kubernetes a permis de gérer efficacement le déploiement, la disponibilité et l'exposition de l'application, tandis que la mise en place d'un pipeline CI/CD avec GitLab a automatisé les différentes étapes, du build de l'image Docker jusqu'au déploiement sur le cluster. Cette approche a permis de réduire les interventions manuelles, d'améliorer la fiabilité du système et de garantir un déploiement rapide et continu.

Au-delà des aspects techniques, ce projet a permis de développer des compétences pratiques essentielles en DevOps, notamment en conteneurisation, orchestration, automatisation d'infrastructure et intégration continue. Il a également mis en évidence l'importance d'une architecture bien pensée, capable de répondre aux exigences de scalabilité, de maintenabilité et de performance dans un contexte réel.

En conclusion, ce projet constitue une **mise en pratique concrète des principes DevOps**, démontrant comment l'automatisation et l'intégration d'outils modernes permettent de construire des systèmes fiables, reproductibles et adaptés aux environnements cloud actuels. Il représente ainsi une base solide pour évoluer vers des architectures plus avancées intégrant sécurité, monitoring et haute disponibilité.

Annexes :

ANNEXE A : Structure du projet :

Dans cette annexe, nous présentons l'organisation globale du projet DevOps, mettant en évidence la séparation des responsabilités entre application, infrastructure et déploiement.

- Structure du projet



Figure 31: structure complete du projet

ANNEXE B : Dockerfile

Cette annexe présente le fichier Dockerfile utilisé pour conteneuriser l'application.

```
swap_app > Dockerfile > ...
1  # Dockerfile pour swap-flask-app
2  FROM python:3.9-slim
3
4  # Installer outils nécessaires
5  RUN apt-get update && apt-get install -y \
6      build-essential \
7      python3-dev \
8      libffi-dev \
9      && rm -rf /var/lib/apt/lists/*
10
11 WORKDIR /app
12
13 # Copier les fichiers
14 COPY requirements.txt /app/
15 COPY . /app/
16
17 # Installer les dépendances
18 RUN pip install --upgrade pip
19 RUN pip install --no-cache-dir -r requirements.txt
20 RUN pip install --no-cache-dir pytest
21 RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
22
23 EXPOSE 5000
24
25 CMD ["python", "run.py", "--host", "0.0.0.0"]
```

Figure 32: Dockerfile

ANNEXE C : Pipeline CI/CD (.gitlab-ci.yml)

Dans cette annexe, nous présentons le fichier .gitlab-ci.yml utilisé pour automatiser les étapes de build, test et déploiement.

```
1  stages:
2    - build
3    - test
4    - deploy
5
6  variables:
7    IMAGE_NAME: "uiaam/swap-flask-app"
8    IMAGE_TAG: "latest"
9    DOCKER_DRIVER: overlay2
10
11  build:
12    stage: build
13    image: docker:20.10.16
14    services:
15      - name: docker:20.10.16-dind
16        command: ["--tls=false"]
17    variables:
18      DOCKER_TLS_CERTDIR: ""
19    script:
20      - echo "Waiting for Docker daemon..."
21      - until docker info >/dev/null 2>&1; do sleep 2; done
22      - echo "Logging in to Docker Hub"
23      - echo "$DOCKER_PASSWORD" | docker login -u "$DOCKER_USER" --password-stdin
24      - echo "Build Docker image"
25      - docker build -t $IMAGE_NAME:$IMAGE_TAG -f swap_app/Dockerfile swap_app
26      - echo "Push Docker image"
27      - docker push $IMAGE_NAME:$IMAGE_TAG
28    only:
29      - main
30
31  test:
32    stage: test
33    image: python:3.9
34    before_script:
35      - pip install --upgrade pip
36      - pip install -r swap_app/requirements.txt
37      - pip install pytest
38      - export PYTHONPATH="$CI_PROJECT_DIR/swap_app:$PYTHONPATH"
39    script:
40      - pytest tests --maxfail=1 --disable-warnings -q
41    only:
42      - main
43
44  deploy:
45    stage: deploy
46    image:
47      name: bitnami/kubectl:latest
48      entrypoint: [""]
49    needs: [build, test]
50    script:
51      - echo "Setup kubeconfig"
52      - mkdir -p ~/.kube
53      - echo "$KUBECONFIG_CONTENT" > ~/.kube/config
54      - export KUBECONFIG=~/.kube/config
55      - echo "Test cluster connection"
56      - kubectl get nodes
57      - echo "Deploy app"
58      - kubectl apply -f kubernetes/deployment.yaml
59      - kubectl apply -f kubernetes/service.yaml
60      - kubectl rollout restart deployment swap-flask-app
61    only:
62      - main
```

Figure 33: Pipeline CI/CD

ANNEXE D : Terraform

Dans cette annexe, nous présentons un exemple de script Terraform utilisé pour provisionner l'infrastructure cloud.

- main.tf

```
terraform > main.tf
 1 resource "aws_instance" "master" {
 2     ami           = "ami-0974a2c5ddf10f442"
 3     instance_type = "t3.micro"
 4     key_name      = "k8s-key"
 5     tags = {
 6         Name = "k8s-master"
 7     }
 8 }
 9
10 resource "aws_instance" "worker" {
11     ami           = "ami-0974a2c5ddf10f442"
12     instance_type = "t3.micro"
13     key_name      = "k8s-key"
14     tags = {
15         Name = "k8s-worker"
16     }
17 }
```

Figure 34: main.tf

ANNEXE E : Ansible

Cette annexe présente un exemple de playbook Ansible utilisé pour configurer les machines et installer Docker.

- inventory

```
ansible > inventory
1 [masters]
2 master1 ansible_host=16.16.65.232 ansible_user=ubuntu ansible_ssh_private_key_file=/root/.ssh/k8s-key.pem
3
4 [workers]
5 worker1 ansible_host=56.228.15.238 ansible_user=ubuntu ansible_ssh_private_key_file=/root/.ssh/k8s-key.pem
6
```

Figure 35: inventory.yml

- Install.yml

```
ansible > ! install.yml
1  - name: Configure servers
2    hosts: all
3    become: yes
4
5    tasks:
6
7      - name: Update apt
8        apt:
9          update_cache: yes
10
11     - name: Install required packages
12       apt:
13         name:
14           - docker.io
15           - python3-pip
16           - curl
17         state: present
18
19     - name: Start Docker
20       service:
21         name: docker
22         state: started
23         enabled: yes
24
25     - name: Add ubuntu to docker group
26       user:
27         name: ubuntu
28         groups: docker
29         append: yes
30
```

Figure 36: install.yml

ANNEXE F : Kubernetes YAML

Dans cette annexe, nous présentons le fichier de déploiement Kubernetes.

- deployment.yaml

```
kubernetes > ! deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: swap-flask-app
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: swap-flask-app
10   template:
11     metadata:
12       labels:
13         app: swap-flask-app
14     spec:
15       containers:
16       - name: swap-flask-app
17         image: uiaam/swap-flask-app:latest
18         ports:
19         - containerPort: 5000
```

Figure 37: deployment.yaml

Cette annexe présente le service permettant d'exposer l'application.s

- service.yaml

```
kubernetes > ! service.yaml
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: swap-flask-app-service
5  spec:
6    type: NodePort
7    selector:
8      app: swap-flask-app
9    ports:
10     - protocol: TCP
11       port: 5000
12       targetPort: 5000
13       nodePort: 30007
```

Figure 38 : service.yaml

ANNEXE H : Variables CI/CD

Cette annexe présente les variables utilisées dans GitLab CI/CD.

DOCKER_PASSWORD 	 	All (default) 	 
mot de passe 2			
Protected			
Masked			
Expanded			
DOCKER_USER 	 	All (default) 	 
DOCKER USER			
KUBECONFIG_CONTENT 	 	All (default) 	 
Protected			

Figure 39: variables CI/CD